

LINUX OS

THEORY ASSIGNMENT 02

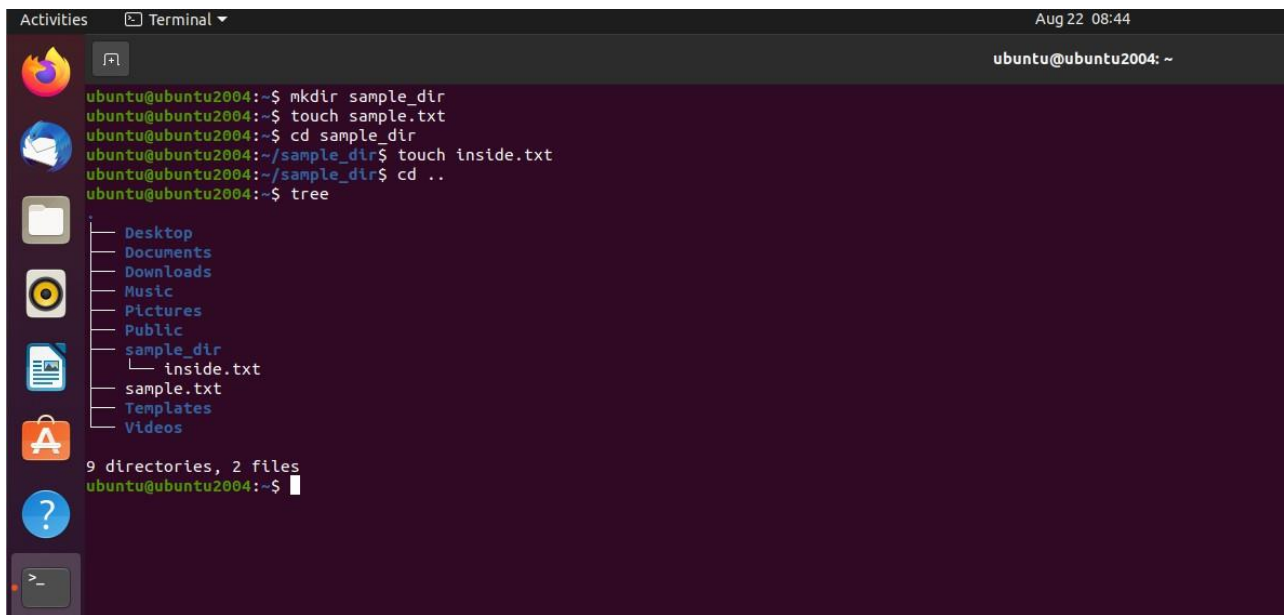
Comparing Permissions on Regular Files and Directories

Student: K P Nidhi
Reg. No.: 261100690024
Date: 22 August 2026

Comparing Permissions on Regular Files and Directories

Objective and setup

The experiment compares owner permissions on a regular file and a directory. It was performed as the normal Ubuntu user `ubuntu`; no `sudo` command was used. The test workspace contained `sample.txt` and `sample_dir/inside.txt`.



```
Activities Terminal Aug 22 08:44
ubuntu@ubuntu2004: ~
ubuntu@ubuntu2004:~$ mkdir sample_dir
ubuntu@ubuntu2004:~$ touch sample.txt
ubuntu@ubuntu2004:~$ cd sample_dir
ubuntu@ubuntu2004:~/sample_dir$ touch inside.txt
ubuntu@ubuntu2004:~/sample_dir$ cd ..
ubuntu@ubuntu2004:~$ tree
.
├── Desktop
├── Documents
├── Downloads
├── Music
├── Pictures
├── Public
├── sample_dir
│   └── inside.txt
├── sample.txt
├── Templates
└── Videos
9 directories, 2 files
ubuntu@ubuntu2004:~$
```

Important distinction: For a directory, `r`, `w`, and `x` apply to the directory’s entries and pathname traversal, not to the contents of each file inside it. This is why an identical numeric mode has different practical effects for a file and a directory.

Permission-test observations

The owner bits were changed with `chmod` before each test. “Read” means using `cat`, “modify” means appending text, “list” means using `ls -la`, “enter” means using `cd`, and “create” means using `touch`.

Permission	Regular file (<code>sample.txt</code>) — test	Observation	Directory (<code>sample_dir</code>) — test	Observation
400 (<code>r--</code>)	Try to read and modify the file	Reading succeeded. Appending/modifying failed with Permission denied because <code>w</code> was absent.	Try to list and enter the directory	Listing failed because directory <code>r</code> was absent. Entering failed because directory <code>x</code> was absent.
200 (<code>-w-</code>)	Try to read and modify the file	Reading failed because <code>r</code> was absent. Appending/modifying succeeded because <code>w</code> was present.	Try to list, enter, and create a file	Listing failed because <code>r</code> was absent; entering failed because <code>x</code> was absent; creating failed because directory <code>x</code> was absent.
100 (<code>--x</code>)	Try to read and execute the file	Reading failed because <code>r</code> was absent. Execution was attempted but failed because <code>sample.txt</code> is a plain text file rather than a valid executable program. The <code>x</code> bit permits execution attempts; it does not turn arbitrary text into a runnable program.	Try to list and enter the directory	Listing failed because <code>r</code> was absent. Entering succeeded because directory <code>x</code> was present.
500 (<code>r-x</code>)	Try to read and modify the file	Reading succeeded. Appending/modifying failed because <code>w</code> was absent. An execution attempt still failed because the file was plain text, not an executable.	Try to list and enter the directory	Listing succeeded because <code>r</code> was present. Entering succeeded because <code>x</code> was present.
600 (<code>rw-</code>)	Try to read and modify the file	Reading and appending/modifying both succeeded because <code>r</code> and <code>w</code> were present.	Try to list, enter, and create a file	Listing could begin, but filenames and metadata could not be resolved reliably because <code>x</code> was absent. Entering failed and creating failed because pathname traversal/search requires <code>x</code> .
700 (<code>rwx</code>)	Try to read, modify, and execute the file	Reading and modifying succeeded. Execution was attempted but failed because <code>sample.txt</code> was plain text rather than a valid executable program; the permission itself was present.	Try to list, enter, create, and delete files	All operations succeeded: the directory could be listed and entered, and a file could be created and deleted.

Terminal evidence

The following screenshots are genuine captures from the Ubuntu VM and show the command transcript for each requested mode.

```
ubuntu@ubuntu2004:~$ chmod 400 sample.txt
ubuntu@ubuntu2004:~$ cat sample.txt
Here is some text.
ubuntu@ubuntu2004:~$ cat >> sample.txt
bash: sample.txt: Permission denied
ubuntu@ubuntu2004:~$ chmod 400 sample_dir
ubuntu@ubuntu2004:~$ ls sample_dir
ls: cannot access 'sample_dir/inside.txt': Permission denied
inside.txt
ubuntu@ubuntu2004:~$ cd sample_dir
bash: cd: sample_dir: Permission denied
ubuntu@ubuntu2004:~$
```

```
ubuntu@ubuntu2004:~$ chmod 200 sample.txt
ubuntu@ubuntu2004:~$ cat sample.txt
cat: sample.txt: Permission denied
ubuntu@ubuntu2004:~$ cat >> sample.txt
This is the second line of text.
ubuntu@ubuntu2004:~$ chmod 200 sample_dir
ubuntu@ubuntu2004:~$ ls sample_dir
ls: cannot open directory 'sample_dir': Permission denied
ubuntu@ubuntu2004:~$ cd sample_dir
bash: cd: sample_dir: Permission denied
```

```
ubuntu@ubuntu2004:~$ chmod 100 sample.txt
ubuntu@ubuntu2004:~$ cat sample.txt
cat: sample.txt: Permission denied
ubuntu@ubuntu2004:~$ ./sample.txt
bash: ./sample.txt: Permission denied
ubuntu@ubuntu2004:~$ chmod 100 sample_dir
ubuntu@ubuntu2004:~$ ls sample_dir
ls: cannot open directory 'sample_dir': Permission denied
ubuntu@ubuntu2004:~$ cd sample_dir
ubuntu@ubuntu2004:~/sample_dir$ cd ..
ubuntu@ubuntu2004:~$
```

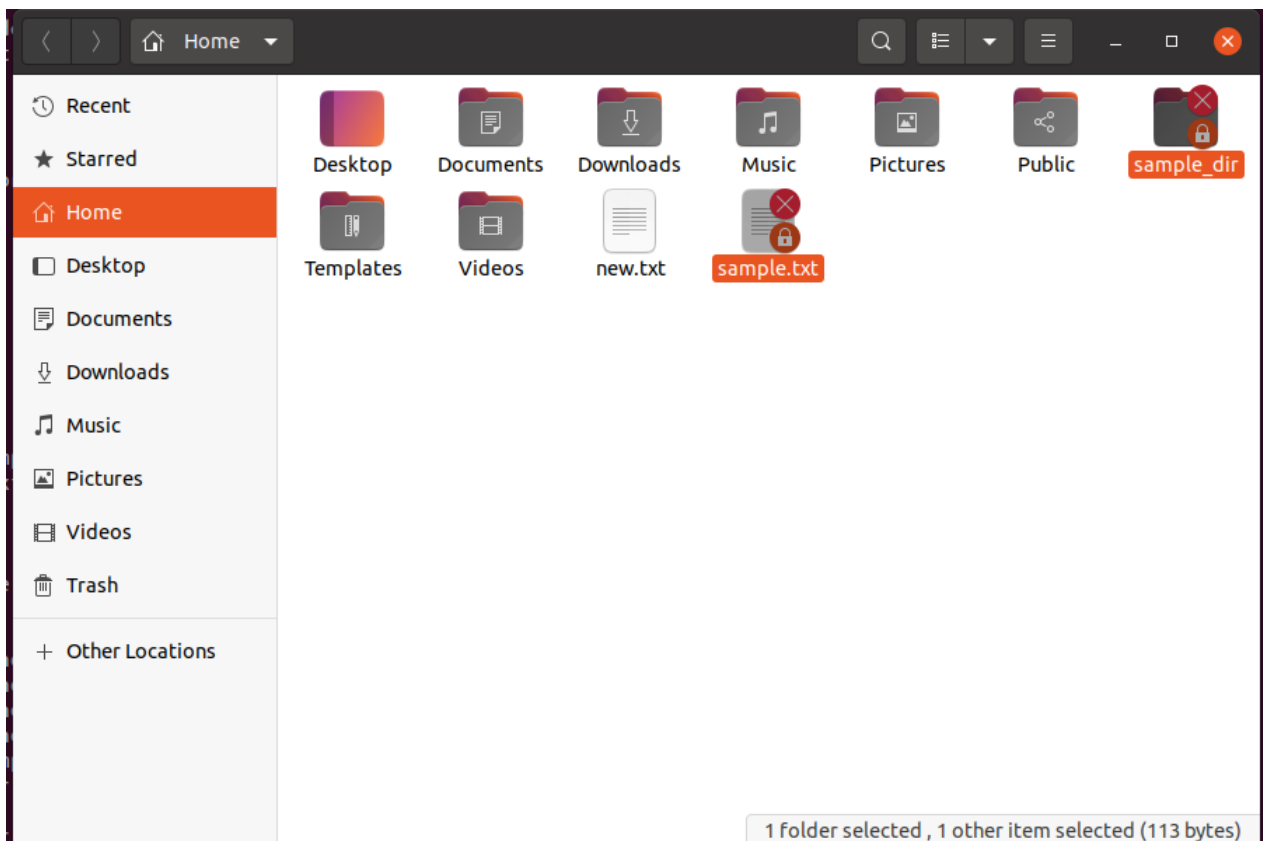
```
ubuntu@ubuntu2004:~$
ubuntu@ubuntu2004:~$ chmod 500 sample.txt
ubuntu@ubuntu2004:~$ cat sample.txt
Here is some text.
This is the second line of text.
ubuntu@ubuntu2004:~$ cat >> sample.txt
bash: sample.txt: Permission denied
ubuntu@ubuntu2004:~$ chmod 500 sample_dir
ubuntu@ubuntu2004:~$ ls sample_dir
inside.txt
ubuntu@ubuntu2004:~$ cd sample_dir
ubuntu@ubuntu2004:~/sample_dir$ cd ..
ubuntu@ubuntu2004:~$
```

```
ubuntu@ubuntu2004:~$ chmod 600 sample.txt
ubuntu@ubuntu2004:~$ cat sample.txt
Here is some text.
This is the second line of text.
ubuntu@ubuntu2004:~$ cat >> sample.txt
Here is another line of text.
ubuntu@ubuntu2004:~$ ./sample.txt
bash: ./sample.txt: Permission denied
ubuntu@ubuntu2004:~$ chmod 600 sample_dir
ubuntu@ubuntu2004:~$ ls sample_dir
ls: cannot access 'sample_dir/inside.txt': Permission denied
inside.txt
ubuntu@ubuntu2004:~$ cd sample_dir
bash: cd: sample_dir: Permission denied
```

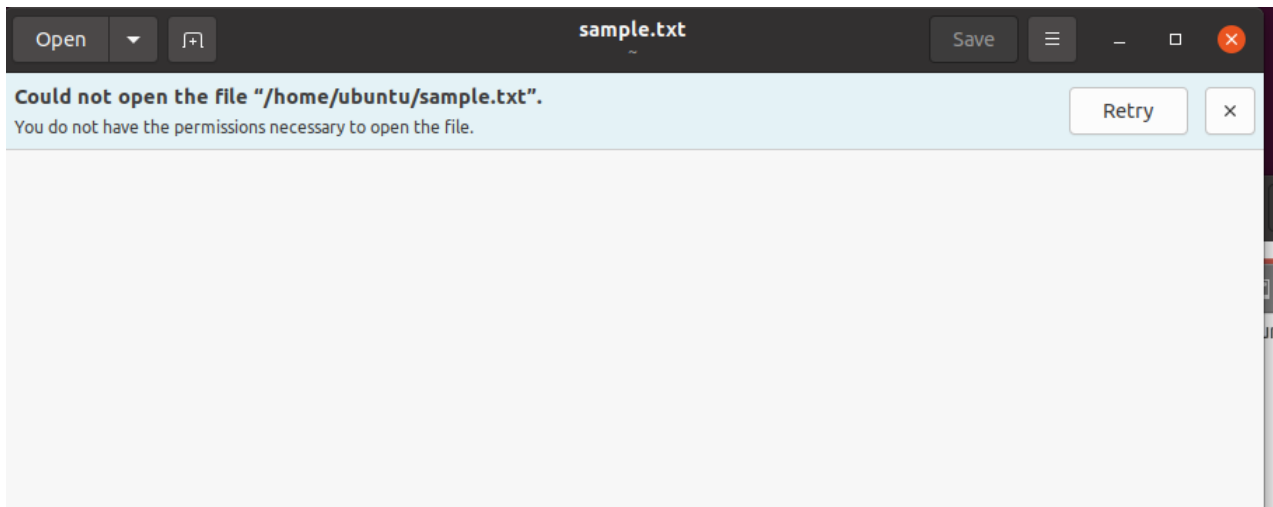
```
ubuntu@ubuntu2004:~$ chmod 700 sample.txt
ubuntu@ubuntu2004:~$ cat sample.txt
Here is some text.
This is the second line of text.
Here is another line of text.
ubuntu@ubuntu2004:~$ cat >> sample.txt
Here is the last line of text.
ubuntu@ubuntu2004:~$ ./sample.txt
./sample.txt: line 1: Here: command not found
./sample.txt: line 2: This: command not found
./sample.txt: line 3: Here: command not found
./sample.txt: line 4: Here: command not found
ubuntu@ubuntu2004:~$ chmod 700 sample_dir
ubuntu@ubuntu2004:~$ ls sample_dir
inside.txt
ubuntu@ubuntu2004:~$ cd sample_dir
ubuntu@ubuntu2004:~/sample_dir$ touch new.txt
ubuntu@ubuntu2004:~/sample_dir$ rm new.txt
ubuntu@ubuntu2004:~/sample_dir$
```

GUI-layer verification

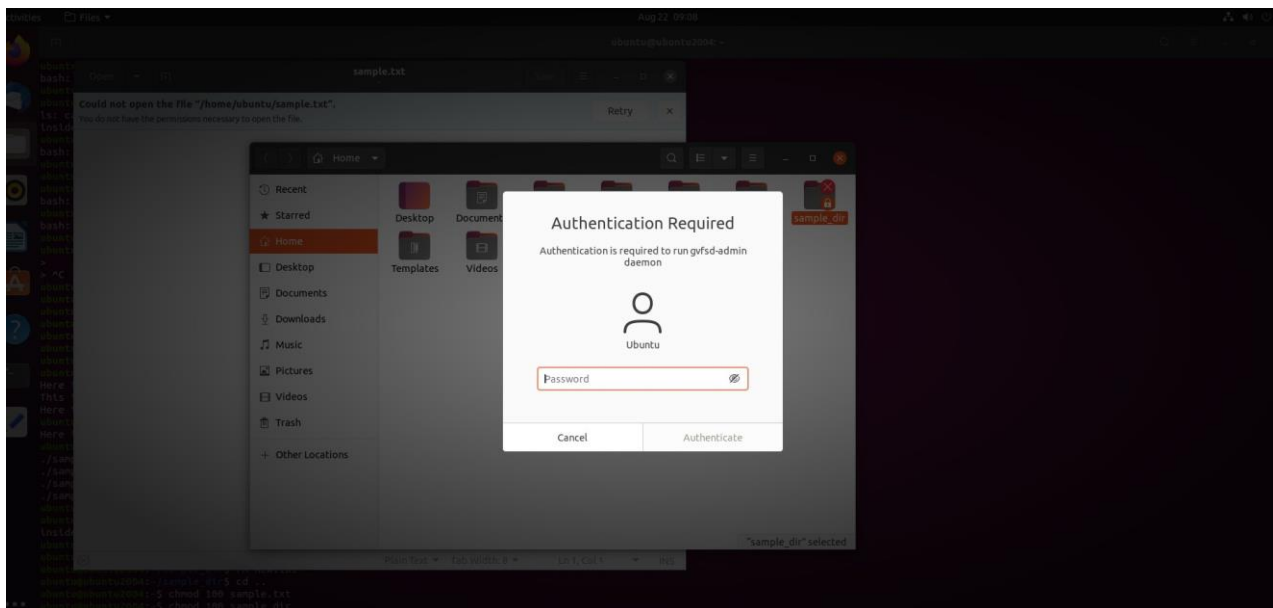
The Ubuntu graphical interface also reflects the permission restrictions observed in the terminal. In the file manager, `sample_dir` and `sample.txt` are visibly marked with lock overlays while restrictive permissions are active.



When the restricted `sample.txt` was opened through the graphical interface, Ubuntu displayed: “Could not open the file ... You do not have the permissions necessary to open the file.” This is consistent with the terminal result that reading was denied when the file lacked `r` permission.



The final GUI screenshot shows Ubuntu requesting authentication for the `gvfsd-admin` daemon. This demonstrates that the graphical layer is applying access-control checks rather than silently bypassing them; it is included as supplementary GUI evidence, not as a substitute for the `chmod` terminal results.



Questions and answers

1. What does read (`r`) permission allow you to do with a regular file?

Read permission allows the file’s contents to be opened and read, for example with `cat`, `less`, or a program. It does not allow the contents to be changed.

2. What does read (**r**) permission allow you to do with a directory?

Read permission allows the directory's entry names to be listed. It does not, by itself, allow entry into the directory or access to the listed files; directory search/traversal generally requires **x** as well.

3. What does write (**w**) permission mean for a regular file?

Write permission allows the file contents to be changed, such as by appending, overwriting, or truncating the file. It does not automatically grant read permission.

4. What does write (**w**) permission mean for a directory?

Write permission allows directory entries to be created, removed, or renamed **when the required search/execute permission is also available**. Directory write permission alone is not sufficient for normal file creation or deletion because the pathname must also be searchable with **x**.

5. What does execute (**x**) permission mean for a regular file?

Execute permission allows the operating system to attempt to run the file as a program. The file must also contain a valid executable format or an appropriate script that can be interpreted. Therefore, **x** alone does not make an ordinary text file executable in practice.

6. Why is execute (**x**) permission important for accessing a directory?

For a directory, **x** means search or traverse: it permits a process to pass through the directory in a pathname and, subject to other permissions, access entries within it. Without **x**, a user may be unable to `cd` into the directory, create files by pathname, or obtain file metadata.

7. Which permission combination allows the owner to fully access a regular file?

700 (rwx -----) gives the owner read, write, and execute permission. In this experiment, it allowed reading and modifying `sample.txt`; the execution attempt failed only because the file was not a valid executable program.

8. Which permission combination allows the owner to fully access a directory?

700 (rwx -----) gives the owner the ability to list the directory, enter/search it, create entries, and delete entries. The directory's contents can still have their own independent permissions.

9. Why do the same **r**, **w**, and **x** permissions behave differently for a regular file and a directory?

The permissions are interpreted according to the type of filesystem object. For a regular file, **r** concerns the file's data, **w** concerns changing that data, and **x** concerns running the file. For a directory, **r** concerns reading the

directory listing, **w** concerns changing its entries, and **x** concerns searching or traversing its pathname.

Consequently, directory operations often require combinations: **r+x** to list and access entries normally, and **w+x** to create or remove entries. The tests demonstrate that `200` can modify a regular file but cannot create a file in a directory, while `100` can permit entering a directory but cannot list it.

Conclusion

The experiment confirms that numeric permission values cannot be interpreted without considering the object type. On regular files, read and write primarily control the data, while execute controls program execution. On directories, read controls listing, write controls changes to the set of entries, and execute controls search/

traversal. The owner mode `700` is the combination that provides the broadest owner access for both objects, although a regular text file still needs valid executable content before it can actually run.